Date.prototype.setFullYear

📄 Título

Date.prototype.setFullYear() – El establecedor del año completo

---

📝 Definición

`Date.prototype.setFullYear()` es un método de JavaScript que establece el **año completo** (de 4 dígitos) de un objeto `Date` según la **hora local**, pudiendo también establecer opcionalmente el mes y el día.

🧠 ¿Qué es?

Es el método estándar para **modificar** el año de un objeto `Date` en JavaScript.

A diferencia de `setYear()` (que está obsoleto y usa un formato de 2 dígitos), `setFullYear()` permite establecer el año **completo de 4 dígitos**, junto con el mes y el día de forma opcional, manteniendo la hora existente.

El método es **mutable**: modifica el objeto `Date` original en lugar de crear uno nuevo.

🎯 ¿Para qué sirve?

- Actualizar el **año** de una fecha existente.
- Crear fechas **modificando solo el año**.
- Implementar **cálculos de fechas** (ej: sumar años).
- Establecer **fechas límite** (expiración, renovaciones).
- Corregir **años incorrectos** en fechas.
- Generar **fechas recurrentes anuales** (aniversarios, cumpleaños).
- Implementar **navegación de calendarios** (cambiar de año).
- Ajustar fechas para **conversiones de zona horaria** (junto con otras funciones).

⚙️ ¿Cómo funciona?

El método modifica el año del objeto `Date` basado en la **hora local** del sistema.

Consideraciones importantes:

- El año debe ser un número de 4 dígitos (ej: 2024).
- Si se proporcionan `month` y `day`, también se actualizan.
- Si no se proporcionan `month` y `day`, se mantienen los valores existentes.
- Los años negativos son válidos (años antes de Cristo).
- El método es **getter/setter** y **modifica** el objeto original.
- El método también actualiza el **timestamp interno** del objeto.

Overflows automáticos:

- Si el mes es mayor que 11, se ajusta automáticamente al año siguiente.
- Si el día es mayor que los días del mes, se ajusta al mes siguiente.
- Los valores negativos se ajustan hacia atrás.

📝 Sintaxis / Estructura (JavaScript)

```
date.setFullYear(year)
date.setFullYear(year, month)
date.setFullYear(year, month, day)
```

Parámetros:

- `year` (obligatorio): Número entero para el año (4 dígitos).
- `month` (opcional): Número entero para el mes (0-11, 0 = enero).
- `day` (opcional): Número entero para el día del mes (1-31).

Retorna:

- El **timestamp** en milisegundos (del objeto modificado).

Errores:

- Si algún parámetro es inválido, el objeto `Date` se vuelve inválido.

💡 Ejemplos

```
// 1. Uso básico - establecer solo el año
const date = new Date(2024, 11, 25, 10, 30);
console.log(date.getFullYear()); // 2024

date.setFullYear(2025);
console.log(date.getFullYear()); // 2025
console.log(date.getMonth()); // 11 (diciembre)
console.log(date.getDate()); // 25
console.log(date.getHours()); // 10 (mantiene la hora)

// 2. Establecer año, mes y día
const date2 = new Date(2024, 5, 15);
date2.setFullYear(2025, 0, 1);
console.log(date2.toISOString()); // 2025-01-01...

// 3. Mantener mes y día existentes
const date3 = new Date(2024, 3, 10, 15, 20);
date3.setFullYear(2030);
console.log(date3.getFullYear()); // 2030
console.log(date3.getMonth()); // 3 (abril)
console.log(date3.getDate()); // 10

// 4. Años negativos (a.C.)
const date4 = new Date(2024, 0, 1);
date4.setFullYear(-500);
console.log(date4.getFullYear()); // -500

// 5. Años con 2 dígitos (NO recomendado)
const date5 = new Date();
date5.setFullYear(99);
console.log(date5.getFullYear()); // 99 (NOTA: no es 1999!)

// 6. Sumar años a una fecha
function addYears(date, years) {
  const newDate = new Date(date);
  newDate.setFullYear(newDate.getFullYear() + years);
```

```
  return newDate;
}

const start = new Date(2024, 0, 1);
console.log(addYears(start, 5)); // 2029-01-01

// 7. Ajuste automático de meses y días (overflow)
const date6 = new Date(2024, 0, 31); // 31 de enero
date6.setFullYear(2024, 1, 31); // 31 de febrero
console.log(date6.getMonth()); // 2 (marzo) - se ajustó aut
omáticamente
console.log(date6.getDate()); // 2 (2 de marzo)

// 8. Establecer año sin modificar mes y día
const date7 = new Date(2024, 6, 20);
date7.setFullYear(2025);
console.log(date7.getFullYear()); // 2025
console.log(date7.getMonth()); // 6
console.log(date7.getDate()); // 20

// 9. Comparación con setYear() (obsoleto)
const date8 = new Date();
date8.setYear(2024); // Obsoleto
console.log(date8.getYear()); // 124 (año - 1900)
console.log(date8.getFullYear()); // 2024 (pero setYear est
á obsoleto)

// 10. Cadena de métodos (retorna timestamp)
const date9 = new Date();
const timestamp = date9.setFullYear(2025);
console.log(timestamp); // Número (timestamp en ms)
console.log(date9.getFullYear()); // 2025

// 11. Valores inválidos resultan en fecha inválida
const date10 = new Date();
date10.setFullYear(NaN);
console.log(isNaN(date10.getTime())); // true
```

```
// 12. Crear fecha para un año específico manteniendo hora
function setYearMaintainTime(date, year) {
  const newDate = new Date(date);
  newDate.setFullYear(year);
  return newDate;
}

const original = new Date(2024, 5, 15, 14, 30);
const modified = setYearMaintainTime(original, 2030);
console.log(modified.getFullYear()); // 2030
console.log(modified.getHours()); // 14
console.log(modified.getMinutes()); // 30

// 13. Validar año antes de establecer
function setYearSafe(date, year) {
  if (!Number.isInteger(year) || year < 0 || year > 9999) {
    throw new Error('Año inválido');
  }
  date.setFullYear(year);
  return date;
}

// 14. Ajuste para fechas bisiestas
function setYearLeapSafe(date, year) {
  const month = date.getMonth();
  const day = date.getDate();

  // Si es 29 de febrero y el año no es bisiesto, ajustar a
l 28 de febrero
  if (month === 1 && day === 29) {
    const isLeap = (year % 4 === 0 && year % 100 !== 0) ||
(year % 400 === 0);
    if (!isLeap) {
      date.setFullYear(year, 1, 28);
      return date;
    }
  }
```

```
  date.setFullYear(year);
  return date;
}

const leapDate = new Date(2024, 1, 29);
console.log(setYearLeapSafe(leapDate, 2023)); // 2023-02-28

// 15. Renovar suscripción anualmente
function renewSubscription(subscription) {
  const newDate = new Date(subscription.expiryDate);
  newDate.setFullYear(newDate.getFullYear() + 1);
  subscription.expiryDate = newDate;
  return subscription;
}

const subscription = {
  userId: 1,
  expiryDate: new Date(2024, 11, 31)
};
renewSubscription(subscription);
console.log(subscription.expiryDate.getFullYear()); // 2025

// 16. Comparar fechas después de modificar
function isInYearRange(date, startYear, endYear) {
  const testDate = new Date(date);
  testDate.setFullYear(startYear);
  const start = testDate.getTime();
  testDate.setFullYear(endYear);
  const end = testDate.getTime();

  return date.getTime() >= start && date.getTime() <= end;
}

// 17. Generar rango de años con fechas
function generateYearRange(startYear, endYear) {
  const dates = [];
  for (let year = startYear; year <= endYear; year++) {
    const date = new Date();
```

```
    date.setFullYear(year);
    dates.push(date);
  }
  return dates;
}
```

⚠️ Errores comunes

- **Usar `setYear()` en lugar de `setFullYear()`:**

```
// ❌ MAL (obsoleto)
date.setYear(2024);

// ✅ BIEN
date.setFullYear(2024);
```

- **Pasar años de 2 dígitos:**

```
// ❌ MAL
date.setFullYear(99); // Establece el año 99 d.C.

// ✅ BIEN
date.setFullYear(1999); // Establece el año 1999
date.setFullYear(2000); // Establece el año 2000
```

- **No considerar meses basados en 0:**

```
// ❌ MAL
date.setFullYear(2024, 12, 1); // Mes 12 -> overflow a e
nero del año siguiente

// ✅ BIEN
date.setFullYear(2024, 11, 1); // Diciembre
```

- **Confundir el orden de parámetros**:

```
// setFullYear(year, month, day)
// NO setFullYear(month, day, year)
```

- **Olvidar que `setFullYear()` modifica el objeto original**:

```
const original = new Date(2024, 0, 1);
const copy = original;
copy.setFullYear(2025);
console.log(original.getFullYear()); // 2025 (¡también m
odificado!)
```

💼 Casos de uso

1. **Sistema de renovación de suscripciones**:

```
class SubscriptionManager {
  constructor() {
    this.subscriptions = new Map();
  }

  createSubscription(userId, plan, startDate = new Date
()) {
    const start = new Date(startDate);
    const duration = plan === 'annual' ? 1 : 0;
    const end = new Date(start);
    end.setFullYear(end.getFullYear() + duration + 1);

    this.subscriptions.set(userId, {
      userId,
      plan,
      start,
      end,
      active: true
```

```
    });

    return this.subscriptions.get(userId);
  }

  renewSubscription(userId) {
    const sub = this.subscriptions.get(userId);
    if (!sub) return null;

    const newEnd = new Date(sub.end);
    newEnd.setFullYear(newEnd.getFullYear() + 1);
    sub.end = newEnd;
    sub.active = true;

    return sub;
  }

  isActive(userId) {
    const sub = this.subscriptions.get(userId);
    if (!sub) return false;

    const now = new Date();
    if (now > sub.end) {
      sub.active = false;
      return false;
    }
    return true;
  }

  getSubscriptionsByYear(year) {
    const result = [];
    for (const [userId, sub] of this.subscriptions) {
      if (sub.start.getFullYear() === year || sub.end.ge
tFullYear() === year) {
        result.push({ userId, ...sub });
      }
    }
    return result;
```

```
  }
}

const manager = new SubscriptionManager();
manager.createSubscription('user123', 'annual', new Date
(2024, 0, 1));
console.log(manager.isActive('user123')); // true
console.log(manager.getSubscriptionsByYear(2024));
```

2. **Sistema de fechas límite y expiración**:

```
class ExpirationManager {
  constructor() {
    this.items = [];
  }

  addItem(name, createdDate = new Date(), validityYears
= 1) {
    const created = new Date(createdDate);
    const expires = new Date(created);
    expires.setFullYear(expires.getFullYear() + validity
Years);

    this.items.push({
      name,
      created,
      expires,
      validityYears,
      isExpired: false
    });

    return this.items[this.items.length - 1];
  }

  checkExpired() {
    const now = new Date();
    this.items.forEach(item => {
```

```
      item.isExpired = now > item.expires;
    });
    return this.items.filter(item => item.isExpired);
  }

  extendValidity(name, additionalYears = 1) {
    const item = this.items.find(i => i.name === name);
    if (!item) return null;

    const newExpiry = new Date(item.expires);
    newExpiry.setFullYear(newExpiry.getFullYear() + addi
tionalYears);
    item.expires = newExpiry;
    item.isExpired = false;

    return item;
  }

  getExpiringSoon(daysThreshold = 30) {
    const now = new Date();
    const threshold = new Date(now);
    threshold.setDate(threshold.getDate() + daysThreshol
d);

    return this.items.filter(item =>
      item.expires >= now && item.expires <= threshold
    );
  }
}

const expManager = new ExpirationManager();
expManager.addItem('Certificado SSL', new Date(2024, 0,
1), 1);
expManager.addItem('Licencia', new Date(2024, 5, 1), 2);
console.log(expManager.getExpiringSoon(365));
```

3. **Generador de fechas de aniversario**:

```
class AnniversaryManager {
  constructor() {
    this.anniversaries = [];
  }

  addAnniversary(name, date, yearOffset = 0) {
    const baseDate = new Date(date);
    this.anniversaries.push({
      name,
      baseDate,
      yearOffset,
      nextDate: this.getNextAnniversary(date)
    });
  }

  getNextAnniversary(date) {
    const now = new Date();
    const anniversary = new Date(date);
    anniversary.setFullYear(now.getFullYear());

    // Si el aniversario de este año ya pasó, ajustar al
próximo año
    if (anniversary < now) {
      anniversary.setFullYear(anniversary.getFullYear()
+ 1);
    }

    return anniversary;
  }

  getUpcomingAnniversaries() {
    const now = new Date();
    const future = new Date(now);
    future.setDate(future.getDate() + 30);

    return this.anniversaries
      .map(a => {
        const nextDate = this.getNextAnniversary(a.baseD
```

```
ate);
        return { ...a, nextDate };
      })
      .filter(a => a.nextDate >= now && a.nextDate <= fu
ture)
      .sort((a, b) => a.nextDate - b.nextDate);
  }

  celebrate(name) {
    const ann = this.anniversaries.find(a => a.name ===
name);
    if (!ann) return null;

    ann.yearOffset++;
    const nextDate = new Date(ann.baseDate);
    nextDate.setFullYear(nextDate.getFullYear() + ann.ye
arOffset);
    ann.nextDate = nextDate;

    return ann;
  }
}

const anniversaries = new AnniversaryManager();
anniversaries.addAnniversary('Cumpleaños Ana', new Date
(1990, 5, 15));
anniversaries.addAnniversary('Aniversario boda', new Dat
e(2015, 8, 22));
console.log(anniversaries.getUpcomingAnniversaries());
```

4. **Sistema de generación de calendarios**:

```
class CalendarGenerator {
  constructor(year = new Date().getFullYear()) {
    this.year = year;
  }
```

```
  setYear(year) {
    this.year = year;
  }

  getMonthDates(month) {
    const date = new Date(this.year, month, 1);
    const days = [];

    while (date.getMonth() === month) {
      days.push(new Date(date));
      date.setDate(date.getDate() + 1);
    }

    return days;
  }

  getYearCalendar() {
    const calendar = {};
    for (let month = 0; month < 12; month++) {
      calendar[month] = this.getMonthDates(month);
    }
    return calendar;
  }

  getDaysByWeekday(weekday) {
    const dates = [];
    const date = new Date(this.year, 0, 1);

    while (date.getFullYear() === this.year) {
      if (date.getDay() === weekday) {
        dates.push(new Date(date));
      }
      date.setDate(date.getDate() + 1);
    }

    return dates;
  }
```

```
  getWeekends() {
    return {
      saturdays: this.getDaysByWeekday(6),
      sundays: this.getDaysByWeekday(0)
    };
  }

  getWeekNumbers() {
    const date = new Date(this.year, 0, 1);
    const weeks = [];
    let week = 1;

    while (date.getFullYear() === this.year) {
      const weekDays = [];
      const weekStart = new Date(date);

      while (date.getDay() !== 0 && date.getFullYear() =
== this.year) {
        weekDays.push(new Date(date));
        date.setDate(date.getDate() + 1);
      }

      if (weekDays.length > 0 || week === 1) {
        weeks.push({ week, days: weekDays });
        week++;
      }

      if (date.getDay() === 0) {
        date.setDate(date.getDate() + 1);
      }
    }

    return weeks;
  }
}

const calendar = new CalendarGenerator(2024);
console.log(calendar.getWeekends());
```

5. **Sistema de facturación con años fiscales:**

```
class FiscalYearManager {
  constructor(fiscalYearStart = 1) { // 1 = enero
    this.startMonth = fiscalYearStart;
  }

  getFiscalYear(date = new Date()) {
    const year = date.getFullYear();
    const month = date.getMonth() + 1;

    if (month >= this.startMonth) {
      return year;
    } else {
      return year - 1;
    }
  }

  getFiscalYearStart(year) {
    const date = new Date(year, this.startMonth - 1, 1);
    return date;
  }

  getFiscalYearEnd(year) {
    const date = new Date(year + 1, this.startMonth - 1,
0);
    return date;
  }

  getQuartersForYear(year) {
    const quarters = [];
    for (let q = 0; q < 4; q++) {
      const startMonth = this.startMonth + q * 3;
      const start = new Date(year, startMonth - 1, 1);
      const end = new Date(year, startMonth + 2, 0);

      quarters.push({
```

```
        quarter: q + 1,
        start,
        end,
        label: `Q${q + 1}${year}`,
        months: Array.from({ length: 3 }, (_, i) =>
          new Date(year, startMonth - 1 + i, 1)
        )
      });
    }
    return quarters;
  }

  getFiscalPeriod(date) {
    const fiscalYear = this.getFiscalYear(date);
    const start = this.getFiscalYearStart(fiscalYear);
    const end = this.getFiscalYearEnd(fiscalYear);
    const current = new Date(date);

    const totalDays = Math.floor((end - start) / (1000 *
60 * 60 * 24));
    const elapsedDays = Math.floor((current - start) /
(1000 * 60 * 60 * 24));

    return {
      fiscalYear,
      start,
      end,
      progress: Math.round((elapsedDays / totalDays) * 1
00),
      daysElapsed: elapsedDays,
      daysRemaining: totalDays - elapsedDays,
      quarters: this.getQuartersForYear(fiscalYear)
    };
  }
}

const fiscal = new FiscalYearManager(1);
```

```
console.log(fiscal.getFiscalPeriod(new Date(2024, 6, 1
5)));
```

6. **Sistema de recordatorios anuales**:

```
class AnnualReminderManager {
  constructor() {
    this.reminders = [];
  }

  addReminder(name, date, recurrence = 'annual') {
    const baseDate = new Date(date);
    this.reminders.push({
      name,
      baseDate,
      recurrence,
      nextDate: this.getNextDate(baseDate)
    });
  }

  getNextDate(baseDate) {
    const now = new Date();
    const next = new Date(baseDate);
    next.setFullYear(now.getFullYear());

    if (next < now) {
      next.setFullYear(next.getFullYear() + 1);
    }

    return next;
  }

  getUpcomingReminders() {
    const now = new Date();
    const future = new Date(now);
    future.setDate(future.getDate() + 30);
```

```
    return this.reminders
      .map(r => ({
        ...r,
        nextDate: this.getNextDate(r.baseDate)
      }))
      .filter(r => r.nextDate >= now && r.nextDate <= fu
ture)
      .sort((a, b) => a.nextDate - b.nextDate);
  }

  markCompleted(name) {
    const reminder = this.reminders.find(r => r.name ===
name);
    if (!reminder) return null;

    const newDate = new Date(reminder.baseDate);
    newDate.setFullYear(newDate.getFullYear() + 1);
    reminder.baseDate = newDate;
    reminder.nextDate = this.getNextDate(newDate);

    return reminder;
  }

  getStats() {
    return {
      total: this.reminders.length,
      upcoming: this.getUpcomingReminders().length,
      reminders: this.reminders.map(r => ({
        ...r,
        daysUntil: Math.floor(
          (this.getNextDate(r.baseDate).getTime() - Dat
e.now()) /
          (1000 * 60 * 60 * 24)
        )
      }))
    };
  }
}
```

```
const reminders = new AnnualReminderManager();
reminders.addReminder('Cumpleaños Juan', new Date(1990,
5, 15));
reminders.addReminder('Aniversario', new Date(2015, 8, 2
2));
console.log(reminders.getUpcomingReminders());
```

7. **Sistema de formación de empleados con certificaciones**:

```
class CertificationManager {
  constructor() {
    this.certifications = [];
  }

  addCertification(employeeId, name, issueDate, validity
Years = 2) {
    const issued = new Date(issueDate);
    const expires = new Date(issued);
    expires.setFullYear(expires.getFullYear() + validity
Years);

    this.certifications.push({
      employeeId,
      name,
      issued,
      expires,
      validityYears,
      active: true
    });

    return this.certifications[this.certifications.lengt
h - 1];
  }

  renewCertification(employeeId, name) {
    const cert = this.certifications.find(
      c => c.employeeId === employeeId && c.name === nam
```

```
e
    );
    if (!cert) return null;

    const newExpiry = new Date(cert.expires);
    newExpiry.setFullYear(newExpiry.getFullYear() + cer
t.validityYears);
    cert.expires = newExpiry;
    cert.active = true;

    return cert;
  }

  getExpired() {
    const now = new Date();
    return this.certifications.filter(c => c.expires < n
ow);
  }

  getExpiringIn(months = 3) {
    const now = new Date();
    const future = new Date(now);
    future.setMonth(future.getMonth() + months);

    return this.certifications.filter(c =>
      c.expires >= now && c.expires <= future && c.activ
e
    );
  }

  getCertificationStatus(employeeId) {
    const certs = this.certifications.filter(c => c.empl
oyeeId === employeeId);
    return certs.map(c => ({
      ...c,
      status: this.getStatus(c.expires)
    }));
  }
```

```
  getStatus(expiryDate) {
    const now = new Date();
    const diff = expiryDate.getTime() - now.getTime();
    const days = Math.floor(diff / (1000 * 60 * 60 * 2
4));

    if (days < 0) return 'expired';
    if (days < 30) return 'expiring_soon';
    if (days < 90) return 'expiring';
    return 'valid';
  }
}

const certManager = new CertificationManager();
certManager.addCertification('emp001', 'JavaScript', new
Date(2023, 0, 1), 2);
certManager.addCertification('emp002', 'React', new Date
(2024, 0, 1), 1);
console.log(certManager.getExpiringIn(12));
```

🧩 Relación con otros conceptos

- `Date.prototype.setFullYear()` **vs** `Date.prototype.setUTCFullYear()`:
  - `setFullYear()`: Establece año en hora local.
  - `setUTCFullYear()`: Establece año en UTC.
- `Date.prototype.getFullYear()`: Obtiene el año.
- `Date.prototype.setMonth()`: Establece el mes.
- `Date.prototype.setDate()`: Establece el día del mes.
- `Date.prototype.setHours()`: Establece la hora.
- `Date.prototype.setMinutes()`: Establece los minutos.
- `Date.prototype.setYear()`: Obsoleto (año - 1900).

🧠 Analogía

Imagina que `setFullYear()` es como un **calendario ajustable**. Tienes una fecha marcada en el calendario y puedes **girar una rueda** para cambiar solo el año, manteniendo el mismo mes y día. Es como si tuvieras una máquina del tiempo que te permite saltar al mismo día y mes de otro año.

📚 Fuente

- MDN Web Docs – Date.prototype.setFullYear()
- ECMAScript Specification – Date.prototype.setFullYear
- MDN – Date